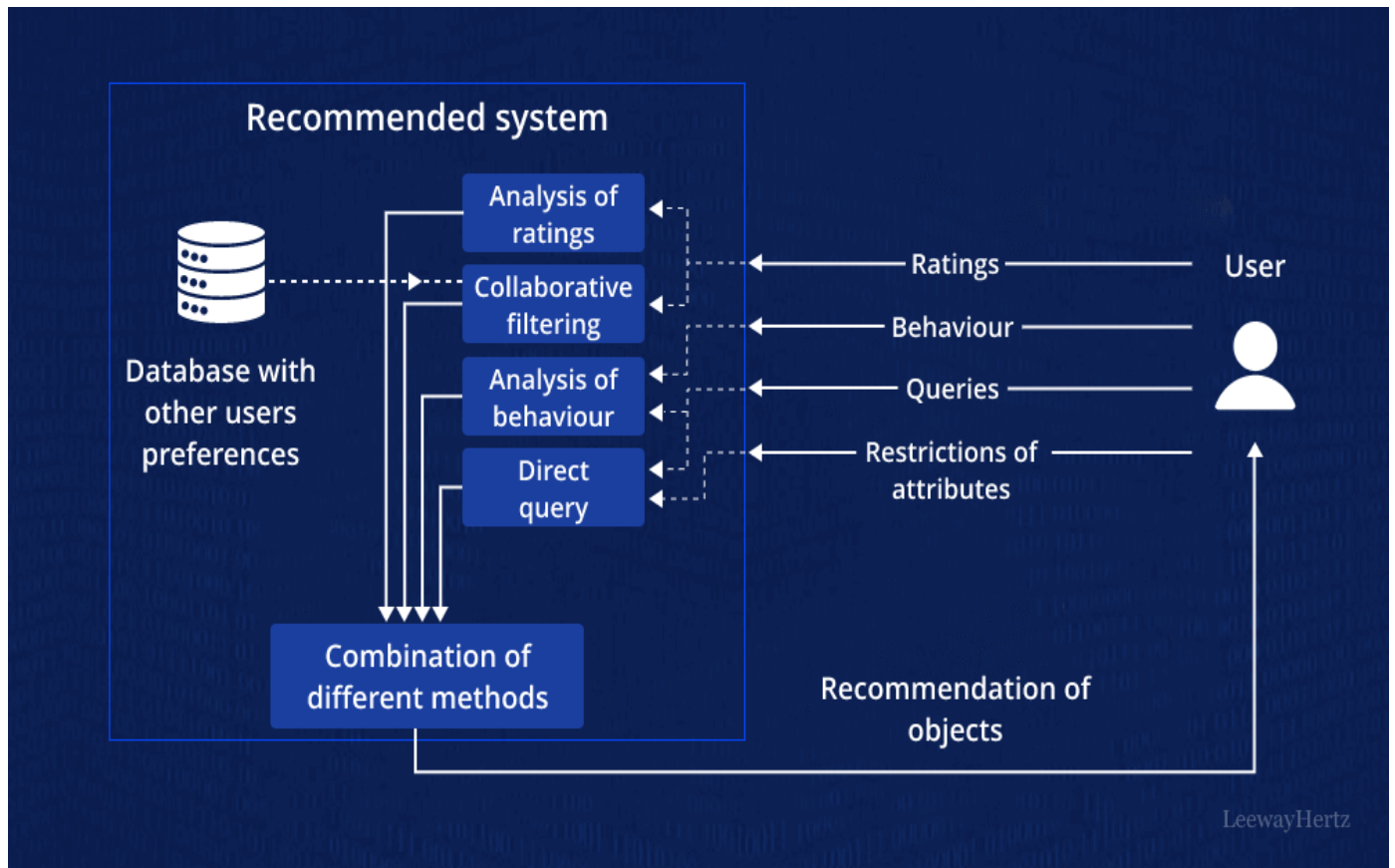


MAJOR PROJECT

NAME: G KARTHIK REDDY

DOMAIN: ARTIFICIAL INTELLIGENCE

Topic- Personalized Recommendation System: Build a recommendation system that suggests products, movies, or music based on user preferences and behavior.



What is a Recommendation System?

A Recommendation System is a filtration program whose prime goal is to predict a user's "rating" or "preference" toward a domain-specific item or item. In our case, this domain-specific item is a movie. Therefore, the main focus of our recommendation system is to filter and predict only those movies that a user would prefer, given some data about the user.

Why Recommendation Systems?

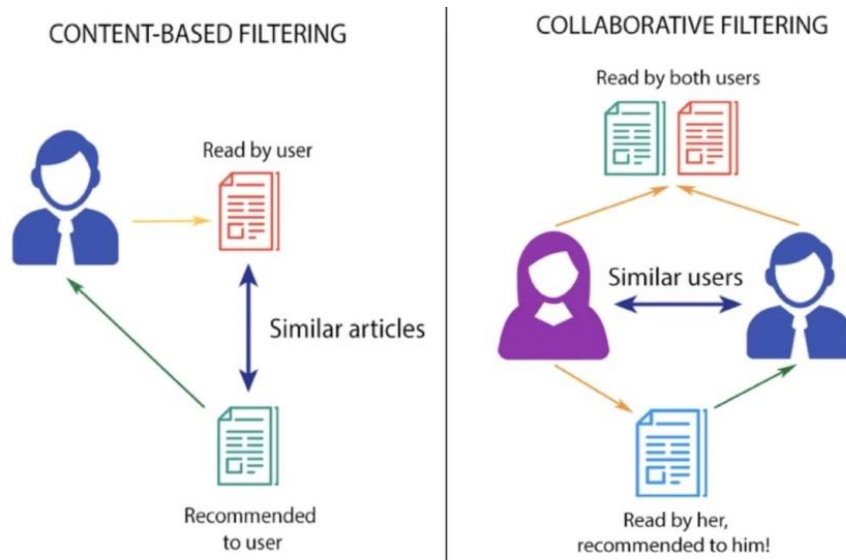
 [Recommendation Systems are essential for several reasons:](#)

1. Recommendation Systems offer personalized suggestions based on user preferences and user ratings, ensuring that they discover content and products that are relevant and interesting to them.
2. By providing tailored recommendations, users are more likely to engage with the platform, increasing user satisfaction and retention.
3. E-commerce platforms like Amazon use recommendation engines to promote products, leading to higher sales and revenue as users discover and purchase items they might not have considered.

4. In today's digital landscape, recommendation systems help users navigate the overwhelming amount of available content, making it easier to find a particular movie or series they seek or in any language they want, like English or Korean.
5. Recommendation algorithms expose users to new and diverse content, expanding their horizons and introducing them to items they might have overlooked.
6. Relying on past behavior and preferences, recommendation systems help users make informed decisions about complex and subjective choices such as movies, music, or books.

Types of Recommendation Systems

There are 2 types of recommendation systems- Content-based filtering and Collaborative Filtering.



Content-based Filtering

Content-based filtering is a recommendation strategy that suggests items similar to those a user has previously liked. It calculates similarity (often using cosine similarity) between the user's preferences and item attributes, such as lead actors, directors, and genres. For example, if a user enjoys 'The Prestige,' the system recommends movies with Christian Bale, the 'Thriller' genre, or films by Christopher Nolan.

Collaborative Filtering

Collaborative filtering is a recommendation strategy that considers the user's behavior and compares it with other users in the database. It uses the history of all users to influence the recommendation algorithm. Unlike a content-based recommender system, a collaborative filtering recommender relies on multiple users' interactions with items to generate suggestions. It doesn't solely depend on one user's data for modeling. There are various approaches to implementing collaborative filtering, but the fundamental concept is the collective influence of multiple users on the recommendation outcome.

There are 2 types of collaborative filtering algorithms:

User-based Collaborative filtering

User-based collaborative filtering aims to recommend items to a user (A) based on the preferences of similar users in the database. It involves creating a matrix of items rated/liked/clicked by each user, computing similarity scores between users, and then suggesting items that user A hasn't encountered yet but similar users have liked. For example, if user A likes particular movies like Batman Begins, Justice League, and The Avengers and user B like Thor, they share similar interests in the superhero genre. Therefore, it is highly likely that user A would enjoy a popular movie like Thor and user B would like The Avengers, one with the highest movie ratings.

Disadvantages:

1. **Fickle User Preferences:** User preferences can change over time, making initial similarity patterns between users outdated. This can result in inaccurate recommendations as users' tastes evolve and they start watching new movies.
2. **Large Matrices:** As the number of users is typically much larger than the number of items, maintaining large matrices becomes challenging and resource-intensive. Regular recomputation is required to keep the data up-to-date.

3. Vulnerability to Shilling Attacks: Shilling attacks involve creating fake user profiles with biased preference patterns to manipulate the recommendation system. User-based collaborative filtering is susceptible to such attacks, potentially leading to biased and manipulated recommendations.

Item-based Collaborative Filtering

Item-based Collaborative Filtering focuses on finding similar movies instead of similar users to recommend to user 'A' based on their past preferences. It identifies pairs of movies rated/liked by the same users, measures their similarity across all users who rated both, and then suggests similar films based on the similarity scores. For example, when comparing movies 'A' and 'B,' we analyze the ratings from users who rated both movies. If these ratings show high similarity, it indicates that 'A' and 'B' are similar movies. Thus, if someone liked 'A', they should be recommended 'B', and vice versa.

Advantages over User-based Collaborative:

Filtering includes stable movie preferences, as popular movies do not change people's tastes. Maintaining and computing matrices is more accessible as there are usually fewer items than users. Shilling attacks are also more challenging since items cannot be faked, making this approach more robust.

Movie Recommendation System Code

In this implementation, when the user searches for a movie, we recommend the top 10 similar movies using our movie recommendation system. We will be using an item-based collaborative filtering algorithm for our purpose. The training set used in this demonstration is the movie lens-small dataset.

First, we need to import libraries which we'll be using in our movie recommendation system with machine Learning. Also, we'll import the dataset by adding the path of the CSV files. Then we will have a look at the movies dataset.

Python Code:

```
[ ] from google.colab import drive
    drive.mount('/content/drive')
```

↗ Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
import pandas as pd
import numpy as np
from scipy.sparse import csr_matrix
from sklearn.neighbors import NearestNeighbors
import matplotlib.pyplot as plt
import seaborn as sns
movies = pd.read_csv('/content/drive/My Drive/Colab Notebooks/movies.csv')
ratings = pd.read_csv('/content/drive/My Drive/Colab Notebooks/ratings.csv')
movies.head()
```

	movieId	title	genres
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	2	Jumanji (1995)	Adventure Children Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama Romance
4	5	Father of the Bride Part II (1995)	Comedy

The movie dataset has the following:

movieId: Once the recommendation is made, we get a list of all similar movieIds and the title for each movie from this dataset.

Genres: Which are not required for this filtering approach.

ratings.head()

	userId	movieId	rating	timestamp
0	1	296	5.0	1147880044
1	1	306	3.5	1147868817
2	1	307	5.0	1147868828
3	1	665	5.0	1147878820
4	1	899	3.5	1147868510

Ratings dataset has:

userId: Unique for each user.

movieId: We take the movie title from the movie dataset using this feature.

Rating: Each user gives ratings for all the films. Using this, we are going to predict the top 10 similar films.

```
final_dataset = ratings.pivot(index='movieId',columns='userId',values='rating')
final_dataset.head()
```

userId	1	2	3	4	5	6	7	8	9	10	...	601	602	603	604	605	606	607	608	609	610
movieId																					
1	4.0	NaN	NaN	NaN	4.0	NaN	4.5	NaN	NaN	NaN	...	4.0	NaN	4.0	3.0	4.0	2.5	4.0	2.5	3.0	NaN
2	NaN	NaN	NaN	NaN	NaN	4.0	NaN	4.0	NaN	NaN	...	NaN	4.0	NaN	5.0	3.5	NaN	NaN	2.0	NaN	NaN
3	4.0	NaN	NaN	NaN	NaN	5.0	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2.0	NaN	NaN
4	NaN	NaN	NaN	NaN	NaN	3.0	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
5	NaN	NaN	NaN	NaN	NaN	5.0	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	3.0	NaN	NaN	NaN	NaN	NaN	NaN

5 rows × 610 columns


```
final_dataset.fillna(0,inplace=True)
final_dataset.head()
```

userId	1	2	3	4	5	6	7	8	9	10	...	601	602	603	604	605	606	607	608	609	610
movieId																					
1	4.0	0.0	0.0	0.0	4.0	0.0	4.5	0.0	0.0	0.0	...	4.0	0.0	4.0	3.0	4.0	2.5	4.0	2.5	3.0	5.0
2	0.0	0.0	0.0	0.0	0.0	4.0	0.0	4.0	0.0	0.0	...	0.0	4.0	0.0	5.0	3.5	0.0	0.0	2.0	0.0	0.0
3	4.0	0.0	0.0	0.0	0.0	5.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	2.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0	3.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
5	0.0	0.0	0.0	0.0	0.0	5.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	3.0	0.0	0.0	0.0	0.0	0.0	0.0

Removing Noise from the Data

In the real world, ratings are very sparse, and data points are mainly collected from Noisetop-rated movies and highly engaged users. We wouldn't want movies rated by a few users because it's not credible enough. Similarly, users who have rated only a handful of films should also not be considered.

Considering all that and some trial-and-error experimentation, we will reduce the noise by adding filters to the final dataset.

To qualify for a movie, a minimum of 10 users should hoist a film. The user should have voted for at least 50 movies to qualify users.

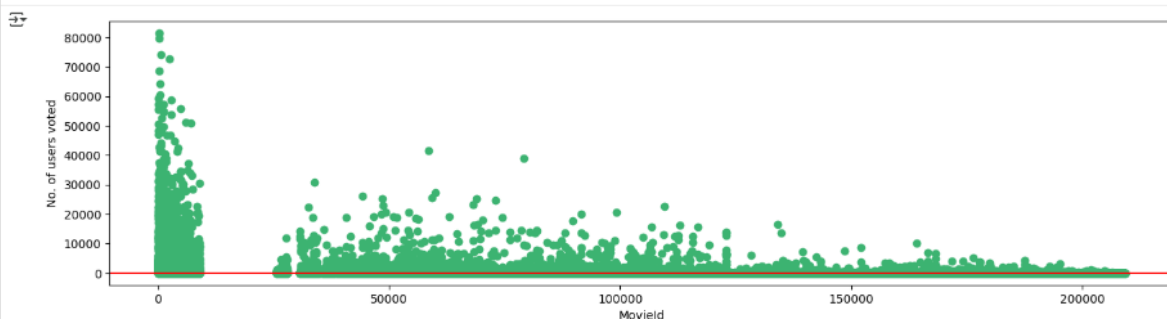
```
[16]
✓ 25 from scipy.sparse import csr_matrix
import pandas as pd

# Map movieId and userId to continuous indices
ratings['movie_index'] = ratings['movieId'].astype('category').cat.codes
ratings['user_index'] = ratings['userId'].astype('category').cat.codes

# Create sparse matrix (movies x users)
final_dataset = csr_matrix((ratings['rating'], (ratings['movie_index'], ratings['user_index'])))
```

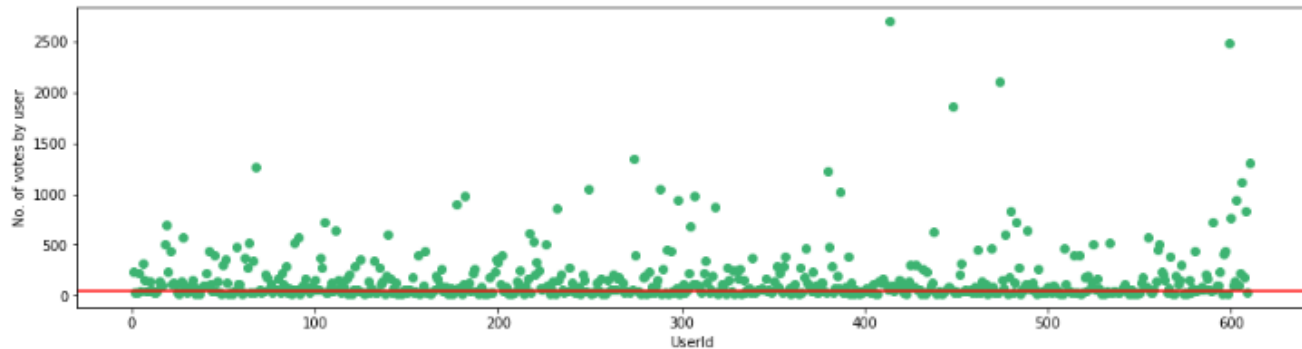
```
[17]
✓ 35 no_user_voted = ratings.groupby('movieId')['rating'].agg('count')
no_movies_voted = ratings.groupby('userId')['rating'].agg('count')

f,ax = plt.subplots(1,1,figsize=(16,4))
# ratings['rating'].plot(kind='hist')
plt.scatter(no_user_voted.index,no_user_voted,color='mediumseagreen')
plt.axhline(y=10,color='r')
plt.xlabel('MovieId')
plt.ylabel('No. of users voted')
plt.show()
```



```
] final_dataset = final_dataset.loc[no_user_voted[no_user_voted > 10].index,:]
```

```
] f,ax = plt.subplots(1,1,figsize=(16,4))
plt.scatter(no_movies_voted.index,no_movies_voted,color='mediumseagreen')
plt.axhline(y=50,color='r')
plt.xlabel('UserId')
plt.ylabel('No. of votes by user')
plt.show()
```



```
: final_dataset=final_dataset.loc[:,no_movies_voted[no_movies_voted > 50].index]
final_dataset
```

```
:  userid  1   4   6   7  10  11  15  16  17  18  ...  600  601  602  603  604  605  606  607  608  610
movieid
1  4.0  0.0  0.0  4.5  0.0  0.0  2.5  0.0  4.5  3.5  ...  2.5  4.0  0.0  4.0  3.0  4.0  2.5  4.0  2.5  5.0
2  0.0  0.0  4.0  0.0  0.0  0.0  0.0  0.0  0.0  3.0  ...  4.0  0.0  4.0  0.0  5.0  3.5  0.0  0.0  2.0  0.0
3  4.0  0.0  5.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  ...  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  2.0  0.0
5  0.0  0.0  5.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  ...  2.5  0.0  0.0  0.0  3.0  0.0  0.0  0.0  0.0  0.0
6  4.0  0.0  4.0  0.0  0.0  5.0  0.0  0.0  0.0  4.0  ...  0.0  0.0  3.0  4.0  3.0  0.0  0.0  0.0  0.0  5.0
...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...
174055  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  ...  0.0  4.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
176371  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  ...  0.0  4.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
177765  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  ...  0.0  4.5  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
179819  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  ...  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
187593  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  ...  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
```

Removing Sparsity

Our final dataset has dimensions of 2121 * 378, and most of the values are sparse. We are using only a small dataset, but for the original large dataset of movie lenses, which has more than 100,000 features, our system may run out of computational resources when fed to the model. We use the `csr_matrix` function from the `scipy` library to reduce the sparsity.

```
csr_data = csr_matrix(final_dataset.values)
final_dataset.reset_index(inplace=True)
```

Making the Movie Recommendation System

We will be using the `knn` algorithm to compute similarity with cosine distance metric which is very fast and more preferable than pearson coefficient.

```
knn = NearestNeighbors(metric='cosine', algorithm='brute', n_neighbors=20, n_jobs=-1)
```

```
knn = NearestNeighbors(metric='cosine', algorithm='brute', n_neighbors=20, n_jobs=-1)
knn.fit(csr_data)
```

```
NearestNeighbors(algorithm='brute', leaf_size=30, metric='cosine',
                  metric_params=None, n_jobs=-1, n_neighbors=20, p=2,
                  radius=1.0)
```

Making the Recommendation Function

The working principle is very simple. We first check if the movie name input is in the database. If it is, we use our recommendation system to find similar movies, sort them based on their similarity distance, and output only the top 10 movies with their distances from the input movie.

```
[24] ✓ Os
def get_movie_recommendation(movie_name, n_movies_to_reccomend=10):
    # find movies that match
    movie_list = movies[movies['title'].str.contains(movie_name, case=False, regex=False)]
    if len(movie_list) == 0:
        return "No movies found. Please check your input"

    # get the first matching movieId
    movie_id = movie_list.iloc[0]['movieId']

    # get the row index of this movie in final_dataset
    if movie_id not in movieId_to_index:
        return "Movie not found in dataset."
    movie_idx = movieId_to_index[movie_id]

    # find nearest neighbors
    knn = NearestNeighbors(metric='cosine', algorithm='brute')
    knn.fit(final_dataset)
    distances, indices = knn.kneighbors(final_dataset[movie_idx], n_neighbors=n_movies_to_reccomend+1)

    # prepare recommendations (skip the first one, since it's the same movie)
    recommend_frame = []
    for i, (idx, dist) in enumerate(zip(indices.squeeze().tolist(), distances.squeeze().tolist())):
        if idx == movie_idx: # skip the movie itself
            continue
        rec_movie_id = index_to_movieId[idx]
        title = movies[movies['movieId'] == rec_movie_id]['title'].values[0]
        recommend_frame.append({'Title': title, 'Distance': dist})

    df = pd.DataFrame(recommend_frame, index=range(1, len(recommend_frame)+1))
    return df
```

Recommendations of some movies

```
get_movie_recommendation('Iron Man')
```

	Title	Distance
1	Up (2009)	0.368857
2	Guardians of the Galaxy (2014)	0.368758
3	Watchmen (2009)	0.368558
4	Star Trek (2009)	0.366029
5	Batman Begins (2005)	0.362759
6	Avatar (2009)	0.310893
7	Iron Man 2 (2010)	0.307492
8	WALL-E (2008)	0.298138
9	Dark Knight, The (2008)	0.285835
10	Avengers, The (2012)	0.285319

```
get_movie_recommendation('Memento')
```

	Title	Distance
1	Iron Man (2008)	0.435163
2	Pan's Labyrinth (Laberinto del fauno, El) (2006)	0.432097
3	Catch Me If You Can (2002)	0.429009
4	Memento (2000)	0.420604
5	Inception (2010)	0.417037
6	Avatar (2009)	0.416482
7	V for Vendetta (2006)	0.410542
8	Dark Knight, The (2008)	0.409689
9	Batman Begins (2005)	0.376506
10	Departed, The (2006)	0.355723

Conclusion

In conclusion, building a movie recommendation system with machine learning significantly enhances user experience by providing personalized movie suggestions based on individual preferences. Utilizing algorithms such as collaborative filtering and content-based filtering, these systems leverage user behavior data to deliver accurate and relevant recommendations. With the integration of Python and robust datasets like MovieLens, developers can create sophisticated recommendation engines akin to Netflix's or Amazon Prime's, using techniques like cosine similarity and matrix factorization. Embracing these technologies enriches user engagement and drives retention and satisfaction, showcasing the transformative power of data science in personalized content delivery.

References

-  Google
-  ChatGpt